LaTeX-on-Lambda

Scaling Document Generation for the Residential Pivot

Discussion Reference

The Worx Company & Client
Prepared by Kurt Vanderwater

2026-08-13

---

*Meeting context: current LaTeX-on-Docker monolith needs to move to a scalable execution model to support the residential-market pivot. This document aggregates prior planning, catalogs execution options, and gives the room shared vocabulary and comparable numbers for the discussion. Tone is deliberately **neutral** — no vendor pushing a recommendation.*

Contents

1 Executive Summary

1.1 The situation

LaTeX-generated documents are currently produced by a Dockerized `pdflatex/latexmk` runtime that lives on the client's single production server (32 GB RAM, running nginx + PHP + MySQL + the LaTeX container). For the current **commercial-only** order volume (~7 orders/day), the monolithic setup is comfortable.

1.2 Why we're revisiting

The client's **residential pivot** projects a target volume of **20,000 sales per month** in a fully-automated flow — roughly 100× current volume, with real concurrency requirements. The existing monolithic execution model is a poor fit for that shape of load. Separately, the operations side (WorxCo) is completing a move to a scalable, resilient AWS architecture that eliminates any single-server aggregations by design. The two forces converge on the same question: *where should LaTeX execution live?*

1.3 What has already been done

Two years ago, an initial **MVP of a custom Lambda container image** for LaTeX generation was built by a member of the client team. It worked end-to-end but was never taken to production because the residential pivot wasn't yet a business priority. That MVP is real institutional knowledge — this discussion is not starting from zero.

1.4 What has changed in two years

Three material shifts:

- **Modern `latexmk` auto-iterates passes.** No more specifying "compile 2, 3, or 4 times to resolve TOC/indexes/tables." A single invocation runs to convergence.
- **AWS Lambda container images matured.** 10 GB image size limit, 10 GB ephemeral storage, 10 GB memory — all comfortably fit a full TeX Live installation plus generation temp space.
- **New AWS execution options emerged**, notably App Runner (2021) and Fargate Spot (2020). Both are relevant to a re-evaluation of the "serverless LaTeX" design space.

1.5 What this document does not do

It does *not* pick a winner. Chapter 5 presents the realistic options (Lambda, Fargate, Batch, App Runner, dedicated EC2), chapter 6 offers a set of **selection criteria** to evaluate against your team's own priorities, and chapters 7–9 give shared numbers and considerations for the group to weigh.

2 Current State

2.1 Production monolith

A single production server currently hosts every component of the platform:

- nginx (web front-end)
- PHP-FPM (Drupal application server)
- MySQL (database)
- Docker container running the LaTeX toolchain
- 32 GB RAM, sufficient for current commercial load

Because everything shares one memory pool, the LaTeX container can burst to 4–6 GB per document without competing with the web tier's normal working set. This works because the concurrency is naturally low.

2.2 Current volume

Order volume from the past three days: 8, 7, 6 — a running average of **7 orders/day**. Peak simultaneous LaTeX generations rarely exceeds one at a time. The Docker container has all the memory it wants because nothing else is asking.

2.3 Two Drupal routes that generate documents

The Drupal codebase invokes document generation from two distinct routes, both discovered during operations testing:

- `/zoning-info/conformance-report/{id}/latex-preview` — custom `report` module; calls `LatexGenerator->renderTemplate()`; `.tex.twig` templates strongly imply the LaTeX toolchain
- `/print/pdf/commerce_order/{id}` — provided by `entity_print` module (8.x-2.18) with the DomPDF engine (pure PHP, no external binary)

Both routes generate synchronously in-request. On the small sandbox PHP instances (which deliberately do NOT have LaTeX installed — adding it "would have far too much of a memory impact on these small machines"), both return HTTP 504 Gateway Timeout after ~60 seconds.

2.4 Why the current architecture works today

Two factors make the monolith comfortable for now:

1. Extreme headroom (32 GB) absorbs LaTeX's memory spikes
2. Low concurrency (single-digit orders/day) means requests essentially don't queue

Neither factor holds under the residential pivot. Chapter 4 unpacks why.

3 Business Driver: The Residential Pivot

3.1 Target volume

Client's residential-market plan projects up to **20,000 sales per month** in a fully-automated flow. That's:

Per month	20,000
Per day (uniform)	~667
Per hour (uniform)	~28
Per business-hour day (8-hr peak)	~84/hr
Peak burst (10% of daily in worst hour)	~67/hr

3.2 Concurrency implications

Residential automation typically clusters — customers act on real-estate transactions during business hours, with peaks around morning and end-of-day. A conservative planning assumption is that peak-hour throughput is **2–4**× the daily-uniform rate. That puts the design point somewhere between **50–120 documents per hour**, or one every 30–70 seconds on average.

If any given document takes 20–45 seconds of LaTeX compute time, then at peak the system needs to support **multiple concurrent generations** without queuing latency ballooning.

3.3 Business consequences of getting this wrong

- **Automation stalls** → orders sit in "pending PDF" state; customer perception of slow product
- **Load-driven timeouts** → the web tier itself degrades when LaTeX consumes shared resources under burst
- **Over-provisioning cost** → solving concurrency by growing a single server means paying for peak capacity 24/7 while using ~5% of it

The right architecture handles these three simultaneously.

4 Why "Just Grow the Box" Fails at Scale

4.1 The memory math

A single LaTeX compile of a typical zoning-info conformance report consumes ~2–4 GB of resident memory during execution (varies with images, table complexity, and index depth). On a 32 GB monolith:

Concurrent generations	Peak memory pressure
1 (today's commercial)	4 GB (comfortable)
5	20 GB (tight; nginx + PHP + MySQL competing)
10	40 GB (does not fit; OOM territory)
20	Guaranteed OOM

At the residential target volume, sustained 5–10 concurrent generations is baseline, not peak. The monolith cannot host this even after being sized up — the shared-memory model itself is the failure.

4.2 The web-tier collateral damage

When LaTeX blocks PHP-FPM workers for tens of seconds, the FPM pool exhausts. *All* incoming requests degrade, not just the ones asking for a PDF. Customer-facing latency rises across the entire product.

4.3 The 24/7-peak-cost problem

The residential load is bursty (business hours, day-of-week seasonality, marketing-event spikes). Growing a monolith to handle peak burst means paying for peak capacity around the clock. In a serverless or auto-scaling model, cost tracks utilization.

4.4 Deferred backlog reference

This project's internal backlog explicitly captures the "don't grow the boxes" position with an operator note:

> "Every 504-fix increase (instance size, ALB timeout, PHP memory limit, DomPDF resource limits) leaves the base architecture wrong: a synchronous request handler blocking a web worker for tens of seconds while it generates a document. Under real load, even bigger instances FPM-pool-exhaust when many users hit 'View PDF' at once." — *from* `docs/memory/pdf-generation-lambda-candidates.md`

5 Options: Where LaTeX Can Live

Five realistic execution models — each has a section in the appendices with technical specifics. This section gives them equal-footing overview and a side-by-side matrix.

5.1 Option A: AWS Lambda with container image

Custom Docker image (up to 10 GB) that contains TeX Live, invoked per-document via SQS trigger or synchronous invoke. Auto-scales to concurrency, zero infrastructure when idle. See Appendix A.

5.2 Option B: AWS Fargate (ECS or EKS)

Long-running container tasks that accept LaTeX jobs from an SQS queue. No cold starts (already-warm workers), no time limit, more control over persistent state. See Appendix B.

5.3 Option C: AWS Batch on Fargate

Purpose-built for queued compute workloads. Auto-provisions Fargate capacity based on queue depth, tears down when idle. Job-oriented, not request-oriented. See Appendix C.

5.4 Option D: Dedicated EC2 Auto Scaling Group with LaTeX AMI

Long-lived EC2 instances baked with TeX Live in an AMI, sitting behind an internal ALB or SQS queue. Full control, familiar operational model. See Appendix D.

5.5 Option E: AWS App Runner

Container-based fully-managed service. Simpler than Fargate to operate (no ECS cluster), auto-scales to zero (when configured). Newer, less mature. See Appendix E.

5.6 Comparison Matrix

Dimension	**Lambda**	**Fargate**	**Batch**	**EC2 ASG**	**App Runner**
Max execution time	15 min	unlimited	unlimited	unlimited	unlimited
Max memory	10 GB	120 GB	120 GB	any	4 GB
Cold start	1–10 s	seconds (task launch)	minutes (queue+launch)	instant (warm)	seconds
Scales to zero	**yes**	no (min 1 task)	**yes**	no	**yes**
Concurrency model	per-invoke	tasks	jobs	per-worker	per-instance
Ops complexity	low	medium	medium-high	high	**lowest**
LaTeX image fit	**excellent** (10 GB)	excellent	excellent	N/A (AMI)	good (4 GB memory limit)
Cost @ 20K/mo	~$15–25/mo	~$60–90/mo	~$40–60/mo	~$50–100/mo	~$40–70/mo
Cost @ 100K/mo	~$80–120/mo	~$80–120/mo	~$70–110/mo	~$50–100/mo	~$60–100/mo

Dimension	**Lambda**	**Fargate**	**Batch**	**EC2 ASG**	**App Runner**
MVP leverage	**high** (image reusable)	medium	medium	low	low
Async / sync fit	**both**	both	async only	both	both
IaC maturity in project	**new build**	**new build**	**new build**	precedent (ASG pattern used elsewhere)	**new build**

All cost figures are order-of-magnitude estimates at us-east-1 pricing, assuming 30-second average generation and 2 GB memory. Chapter 7 unpacks the model.

6 Selection Criteria

Rather than a recommendation, here is a framework for the group to weigh options against actual priorities. Each criterion below has an obvious "winner" among the options — but that only matters if the criterion itself matters to *this* team, on *this* project, at *this* moment.

6.1 Cost fit

Winner at 20K/mo: Lambda (usage-based, tiny at that volume).
Winner at 100K+/mo: EC2 ASG (fixed always-on cheaper than per-invocation).
Weight this if: cost predictability or minimization is a stated goal. Residential is projected but not-yet-real; over-optimizing for volume that hasn't materialized may not pay off.

6.2 MVP leverage

Winner: Lambda (the 2-year-old custom-image MVP is directly reusable structurally; the image contents need modernization but the deployment shape is done).
Weight this if: reducing time-to-production matters more than picking the theoretical best.

6.3 Operational simplicity

Winner: App Runner (fewest moving parts). Lambda close second (no cluster, no VPC required by default).
Loser: EC2 ASG (AMI lifecycle, patching, capacity planning).
Weight this if: the ops team is small or already stretched.

6.4 Cold-start tolerance

Winner: EC2 ASG (always warm). Fargate close second (task-level warm-pool).
Loser: Lambda (1–10s cold start on a fresh container).
Weight this if: customer-facing UX requires sub-second document appearance. *Doesn't matter if* the flow is async (email me the PDF).

6.5 Elasticity / peak absorption

Winner: Lambda (thousands of concurrent invocations available in the region-wide account quota, scales to peak in seconds).
Loser: EC2 ASG (minutes to scale up new instances).
Weight this if: expected traffic patterns include sharp spikes (marketing events, batch imports, monthly runs).

6.6 Runtime limits

Losers: Lambda (15-min hard cap; if any document ever takes longer, it fails).
Winners: Fargate, Batch, EC2 ASG (no time limit).
Weight this if: any generated document might exceed 15 minutes of compute — rare for LaTeX but possible for large, image-heavy reports with many index-resolution passes.

6.7 Integration with existing WorxCo IaC

Precedent: EC2 ASG (the project already uses ASG-fronted architecture for nginx and PHP-FPM tiers).
New territory: everything else.
Weight this if: consistency of infrastructure patterns across the platform is valued over per-workload optimization.

6.8 Vendor lock-in gradient

Most portable: EC2 ASG (a Linux VM running LaTeX — runs anywhere).
Least portable: Lambda (deeply Lambda-specific handler signature; container-image approach softens this significantly).
Weight this if: multi-cloud or on-prem migration is on any horizon.

7 Cost Projections

7.1 Assumptions

- Average LaTeX compilation: 30 seconds, 2 GB memory
- us-east-1 pricing
- arm64 (Graviton) where the service supports it (Lambda, Fargate, EC2)
- Fargate: 2 tasks running continuously for baseline; Fargate Spot @ 30–50% list price
- EC2 ASG: 2 × m6g.large baseline (2 vCPU, 8 GB), always-on
- App Runner: 2 vCPU / 2 GB configuration, provisioned + active mix
- No egress charges included (small PDFs, all within AWS to S3)

7.2 Cost at various volumes

Service	1K/mo	20K/mo	100K/mo	500K/mo
Lambda (arm64)	$0.80	$16	$80	$400
Fargate (2 always-on)	$65	$65	$90	$220
Fargate Spot (2 always-on)	$25	$25	$40	$120
Batch on Fargate	$5	$45	$100	$350
EC2 ASG (2 m6g.large)	$110	$110	$110	$150
App Runner (2 vCPU baseline)	$40	$50	$80	$240

7.3 Reading the cost curves

Three distinct cost shapes visible in the table:

- **Linear-with-usage** (Lambda, Batch, App Runner): near-zero at low volume, scale up cleanly with real load. Best when volume is uncertain or bursty.
- **Flat-then-linear** (Fargate, Fargate Spot): base cost for keeping tasks warm; small marginal cost per additional job. Best when steady-state utilization justifies the baseline.
- **Nearly-flat** (EC2 ASG): fixed cost dominates until you exceed baseline capacity. Cheapest at very high volume, most expensive at low.

7.4 The crossover points

At the projected 20K/month: Lambda is the clear cost leader, but the delta vs Fargate Spot ($16 vs $25) is not decisive.
At 100K/month: pricing converges into a $80–$120 band; other criteria dominate.
At 500K/month: Lambda's per-invocation scaling starts to price it above always-on EC2.

8 From MVP to Production

Any option chosen will need to bridge from "a working prototype" to "a production system that carries residential-order traffic." This chapter enumerates the work regardless of option.

8.1 Toolchain modernization

Two years of TeX Live releases and `latexmk` improvements to bring forward:

- **Auto-iterating passes**: single `latexmk -pdf -interaction=nonstopmode` call replaces the previous "specify 2, 3, or 4 passes" logic
- **TeX Live 2024/2025**: current package ecosystem, security fixes since the MVP era
- **Font handling**: if the templates use non-Latin scripts or ligature-heavy fonts, XeTeX or LuaTeX (rather than pdflatex) is the modern path

8.2 Integration with Drupal

The two document-generating Drupal routes need to be re-shaped from "synchronous in-request" to "async fire-and-poll" for any external-execution model.

- **Route 1 (LaTeX conformance report)**: `/zoning-info/conformance-report/{id}/latex-preview` — currently invokes `LatexGenerator->renderTemplate()` in-band. New shape: enqueue an SQS job with the report ID + template context, return a job-status page; Drupal polls or receives an SNS callback when the S3 URL is ready.
- **Route 2 (Commerce order PDF)**: `/print/pdf/commerce_order/{id}` — currently uses `entity_print` with the DomPDF engine (pure PHP, no LaTeX). This one might not need LaTeX at all — but it *does* need to move off in-request rendering.

8.3 Storage and delivery

Generated PDFs land in an S3 bucket. Users receive a pre-signed URL for time-limited download. Existing project pattern: `/_flysystem/s3/` route already serves cached S3-hosted PDFs without issue — generated documents would use the same delivery path.

8.4 Observability

- CloudWatch metrics: job duration, failure rate, cold-start percentage (Lambda specifically), queue depth (SQS)
- CloudWatch alarms: p95 duration threshold, error rate threshold, queue-age threshold
- X-Ray traces (Lambda) or OpenTelemetry (Fargate/Batch/EC2) to link Drupal request → enqueue → generation → delivery

8.5 Bundle-class coverage gap (a related pre-existing item)

Prior operations testing surfaced a real Drupal-side issue: `ResearchDocumentBundleBase` defines `getResearchAnswer()`, but 4 of 12 research-document bundles have no subclass and Drupal instantiates the base class (which lacks the method). Two call sites in the `report` module invoke `->getResearchAnswer()` unguarded, and throw when they hit an unimplemented bundle:

- `ReportHooks::latexPreprocess()` line 345
- `ConformanceAnalysisTrait` line 86

Meanwhile, `FannieFieldResolver.php` line 329 uses `method_exists()` defensively and works fine. Either the missing 4 subclasses get implemented, or the two report-side call sites adopt the `method_exists()` guard pattern. This is orthogonal to the Lambda decision but should be resolved before residential launch — otherwise LaTeX generation will throw on documents referencing those bundles regardless of where the LaTeX runs. See `docs/memory/research-document-bundle-gaps.md` for full context.

9 Rollout and Risk Considerations

9.1 Suggested phased approach (option-agnostic)

1. **Pick the execution model** (this discussion's outcome)
2. **Bring the MVP forward**: modernize the LaTeX container / AMI with current TeX Live + `latexmk` auto-passes
3. **Shadow mode**: enqueue jobs from the current monolith but continue serving the in-band Docker output; compare outputs
4. **Sandbox smoke test**: verify end-to-end async flow through Drupal → queue → execution → S3 → presigned URL
5. **Cutover with feature flag**: Drupal route conditionally serves in-band (old) or redirects to async job (new); flag flips per env then per traffic percentage
6. **Retire the Docker-in-monolith LaTeX path**

9.2 Risks common to all options

- **Template drift**: the current Docker container's TeX Live + package versions become the accidental reference; some templates may rely on specific package versions
- **Font / locale surprises**: pre-generated PDFs from prod are the ground-truth for "does it render identically" comparison
- **Cost surprises**: bursty traffic (marketing send, batch imports) can push volume 5–10× over baseline; per-invocation options handle this transparently, always-on options don't
- **Failure modes to design for**: LaTeX compilation errors (bad template), timeouts, S3 upload failures, notification delivery failures — all of which need idempotent retry semantics

9.3 Risks specific to serverless (Lambda, Batch, App Runner)

- Cold starts on infrequently-hit code paths
- Concurrency quotas (Lambda default 1000 concurrent per account, raisable)
- Deployment story for a 5+ GB container image is slower than a small zip

9.4 Risks specific to always-on (Fargate, EC2 ASG)

- Baseline cost when idle
- Patching / AMI refresh cadence
- Manual scale-out response time if peak exceeds pre-provisioned capacity

A Lambda with Container Image

A.1 Fit for LaTeX

Lambda's container-image support (up to 10 GB) accommodates a full TeX Live installation plus the required custom templates and fonts. Ephemeral `/tmp` at 10 GB is generous for intermediate PDF passes and auxiliary files. Memory can be set anywhere from 128 MB to 10 GB (allocated in 1 MB increments); the LaTeX case typically wants 2–4 GB.

A.2 Invocation shape

Two natural patterns:

- **SQS-triggered async**: Drupal enqueues a job, Lambda consumes from SQS, uploads PDF to S3, sends SNS notification (or Drupal polls). Best throughput, best failure isolation.
- **Direct invoke via API Gateway or Function URL**: synchronous request-reply. Simpler, but subject to the 15-minute Lambda ceiling and API Gateway's own 29-second cap (functions URLs are cleaner here, no gateway cap).

A.3 Cold starts

A large container image (multi-GB TeX Live) has cold-start latency in the 5–10 second range on first invocation. Warm invocations start in milliseconds. Provisioned Concurrency pins a chosen number of always-warm instances at additional cost (~$5/mo per always-warm instance at 2 GB memory).

A.4 IAM & networking

- Execution role: read from SQS queue, write to S3 bucket, publish to SNS topic (if used), write to CloudWatch Logs
- VPC: optional; not required unless the function needs to reach a resource inside the VPC (like RDS). Skipping VPC attachment eliminates a class of cold-start latency.
- Concurrency: reserved concurrency to isolate this workload from other Lambdas; account-level limit is 1000 concurrent by default, raisable

A.5 2-year-old MVP relevance

The MVP built two years ago used the same container-image mechanism. The IAM, deployment, and invocation shape all carry forward directly. Only the image *contents* (TeX Live version, template updates, `latexmk` instead of manual pass counting) need modernization.

B AWS Fargate

B.1 Fit for LaTeX

Fargate runs container tasks without managing EC2 or Kubernetes nodes. Task CPU sizing goes up to 16 vCPU, memory up to 120 GB — vastly larger than any single LaTeX document could need. Runs continuously as long as the task is alive.

B.2 Invocation shape

Long-running worker pattern:

1. 2–N Fargate tasks running continuously
2. Each polls an SQS queue for jobs
3. Generates PDF, uploads to S3, ACKs the message

Auto Scaling on queue depth grows/shrinks the task count.

B.3 Fargate Spot

Interruption-tolerant workloads (which LaTeX generation is — a killed task's SQS message returns to the queue for retry) can use Fargate Spot at roughly 30–50% of on-demand pricing.

B.4 Cost baseline

Two continuously-running 2 vCPU / 4 GB tasks on Fargate on-demand: ~$100/month.
Same shape on Fargate Spot: ~$35–50/month.

B.5 Trade-offs vs Lambda

+ No time limit, no memory ceiling (matters if some PDFs go long)
+ No cold starts on warm tasks
– Always-on cost even when queue is empty
– Task management (ECS cluster, service definitions, task definitions) is heavier IaC than a Lambda function

C AWS Batch on Fargate

C.1 Fit for LaTeX

Batch is purpose-built for queued job execution. It provisions Fargate (or EC2) capacity in response to job-queue depth, executes jobs, tears down when idle. Excellent scale-to-zero characteristics.

C.2 Invocation shape

Drupal submits a job to a Batch job queue; Batch schedules it onto a Fargate task from a compute environment; result lands in S3. Batch tracks job state (RUNNABLE, RUNNING, SUCCEEDED, FAILED).

C.3 When Batch wins over Lambda or Fargate

- Job-oriented mental model matches ”generate this PDF” better than ”invoke this function”
- Built-in retry policies, priority queues, dependency graphs (if some documents depend on others)
- Compute environments can be mixed (Fargate for small jobs, EC2 for big ones)

C.4 When Batch loses

- Startup latency: allocating a fresh Fargate task per job adds ~30–60s to first response
- Overkill for a single-workload use case; Batch's job/queue/compute-env abstraction is heavier than needed if only one workload runs on it

D Dedicated EC2 Auto Scaling Group

D.1 Fit for LaTeX

Standard EC2 pattern: a custom AMI baked via Image Builder that includes TeX Live pre-installed. An Auto Scaling Group maintains N instances behind an SQS queue or ALB. WorxCo already uses this pattern for the nginx and PHP-FPM tiers in the current platform.

D.2 Familiarity advantage

The team knows this pattern:

- Image Builder recipe (like the existing nginx / php83 recipes)
- ASG with scaling policies
- CloudWatch alarms driving scale-out / scale-in

D.3 Downsides

- Always-on cost (even minimum ASG size 1 pays 24/7)
- AMI lifecycle: patching, TeX Live version management, testing new AMIs before rolling
- Scale-out latency (2–5 minutes for a new instance to launch, bootstrap, and become healthy) is a real thing during burst

D.4 When this wins

Steady-state volume high enough that always-on is cheaper than per-invocation, AND operational familiarity is worth more than the ceremony cost of learning a serverless service. At the projected 20K/month, both conditions are borderline.

E AWS App Runner

E.1 Fit for LaTeX

App Runner (introduced 2021) is a fully-managed container service — push a container image, App Runner runs it behind a load balancer with auto-scaling. Simpler than Fargate (no ECS cluster to define), simpler than Lambda (standard HTTP handler, no Lambda-specific handler signature).

E.2 Constraints relevant to LaTeX

- Maximum memory per instance: 4 GB (as of Q1 2025 — verify at time of decision)
- Maximum vCPU per instance: 2
- Container image handling is straightforward; ECR-hosted images work out of the box

E.3 Why it's on the list

It's the shortest path from "here is a container image" to "here is a running service." No cluster management, no VPC required, no Auto Scaling Group. If LaTeX generation fits within its per-instance limits, App Runner is the operationally simplest option.

E.4 Why it might not fit

The 4 GB memory ceiling is tight for LaTeX. If any documents ever push past 3–3.5 GB working-set memory, App Runner is off the table. This is a real risk with image-heavy conformance reports.

F The 2024 MVP: What Was Built

F.1 Scope of the MVP

Two years ago (roughly late 2024), a member of the client team built a custom Lambda container image containing a TeX Live installation and a proof-of-concept LaTeX compilation entrypoint. It generated a sample PDF end-to-end from a Lambda invocation.

F.2 Why it didn't ship

At the time, the residential pivot was a lower priority for the client. Commercial order volume (7 orders/day) didn't stress the existing Docker-on-monolith path. Rather than integrate the MVP into production for a workload that didn't need it, the team parked it.

F.3 What still applies today

- The choice of "Lambda container image" as the deployment shape is still valid — Lambda's container support has only improved
- The deployment pipeline (build image, push to ECR, update Lambda function) is well-trodden AWS territory
- The MVP's Dockerfile is a reasonable starting point for a modernized build

F.4 What needs updating

- TeX Live 2024/2025 vs whatever the MVP shipped with
- `latexmk` auto-iteration replacing manual pass-count specification
- Modern Lambda base image (arm64 / Graviton for cost)
- Integration with SQS + S3 + Drupal (never wired up in the MVP)

G Drupal Integration Points

G.1 Route 1: LaTeX conformance report

- URL: `/zoning-info/conformance-report/{id}/latex-preview`
- Module: custom `report` module
- Current: calls `LatexGenerator->renderTemplate()` synchronously in-request
- Template location: `.tex.twig` files in the `report` module
- Uses (see chapter 6): `ReportHooks::latexPreprocess()` line 345 iterates research documents and calls `getResearchAnswer()` on each — ties this route to the bundle-class-coverage gap

G.2 Route 2: Commerce order PDF

- URL: `/print/pdf/commerce_order/{id}`
- Module: `entity_print` (8.x-2.18) with the DomPDF engine
- Current: pure PHP (no LaTeX), still 504s at the 60s mark
- Notable: this one might not need LaTeX at all — but it does need to move off synchronous in-request generation

G.3 Comparison to existing s3-served PDFs

- URL pattern: `/_flysystem/s3/…`
- Behavior: streams pre-existing PDFs from the sandbox media S3 bucket
- Works fine (not part of the problem) — and provides the delivery pattern the new async flow can adopt for freshly-generated documents

G.4 Suggested Drupal-side changes for any option chosen

1. Introduce a `documents_queue` service that submits jobs to SQS (or the equivalent for whichever option is chosen)
2. Replace the current in-request `LatexGenerator->renderTemplate()` call with the queue submission + return of a job-status URL
3. Add a status endpoint the browser polls (or an SNS → email → link callback for truly async delivery)
4. Once ready, the delivered PDF is served via the same `/_flysystem/s3/` pattern already in production

G.5 Feature-flag rollout

The Drupal side can gate this behavior with a config toggle so the async path is enabled per-env (sandbox first, then staging, then a percentage of production traffic) without a hard cutover.

Index